

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

```
110  -- ===== SQL Basics (SELECT, Constraints, Primary Keys) =====
111  •  select * from customers;
112
```

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

```
113  •  select first_name,last_name,city from customers;
114
```

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
115  •  SELECT DISTINCT category FROM products;
116
```

	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

117 • DESCRIBE customers;

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO	MUL	NULL	
state	varchar(50)	NO	MUL	NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

119 • DESCRIBE products;

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
product_name	varchar(100)	NO		NULL	
category	varchar(50)	NO	MUL	NULL	
brand	varchar(50)	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
stock_qty	int	NO		0	

121 • DESCRIBE orders;

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	int	NO	MUL	NULL	
order_date	date	NO	MUL	NULL	
status	varchar(20)	NO	MUL	Pending	
total_amount	decimal(12,2)	NO		NULL	

123 • DESCRIBE order_items;

124

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	
order_id	int	NO	MUL	NULL	
product_id	int	NO	MUL	NULL	
quantity	int	NO		NULL	
unit_price	decimal(10,2)	NO		NULL	
discount_pct	decimal(5,2)	YES		0.00	

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

➤ email VARCHAR(100) UNIQUE NOT NULL

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

INSERT INTO products (product_id, product_name, unit_price) VALUES (101, 'Laptop', -50);

➤ The insertion will fail because the unit_price cannot be negative.

✖ 27 08:54:30 INSERT INTO products (product_id, product_name, unit_price) VALUES (101, 'Laptop', -50) Error Code: 1364. Field 'category' doesn't have a default value

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

127 • `select * from orders where status = 'delivered';`

128

Result Grid



Filter Rows:

Edit: 

E

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

_price greater than ₹2000.

```
129 • SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;
130
```

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
131 • SELECT * FROM customers WHERE YEAR(join_date) = 2024 AND state = 'Maharashtra';
132
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
133 • SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';
134
```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

- The index stores the order_date values in a sorted structure, allowing the database to quickly locate rows based on dates instead of scanning the entire table.

How does it improve performance?

When a query filters records using order_date, the index helps the database:

- search fewer rows
- reduce disk reads
- return results faster

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

135 • `SELECT * FROM customers WHERE YEAR(join_date) = 2024;`

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

No, the index on `join_date` would **usually not be used efficiently** in this query:

Why?

The function `YEAR()` is applied to the indexed column `join_date`.

When a function is used on an indexed column:

- the database must calculate `YEAR(join_date)` for every row
- this prevents efficient index searching
- it may perform a full table scan instead

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

137 • `SELECT COUNT(*) AS total_orders FROM orders;`

total_orders
10

Q14. Find the total revenue (SUM of `total_amount`) from all 'Delivered' orders.

139 • `SELECT SUM(total_amount) AS total_revenue FROM orders`
 140 `WHERE status = 'Delivered';`

total_revenue
17191.00

Q15. Calculate the average `unit_price` of products in each category.

142 • `SELECT category, AVG(unit_price) AS average_price FROM products`
 143 `GROUP BY category;`

category	average_price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```
145 • SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue
146 FROM orders GROUP BY status
147 ORDER BY total_revenue DESC;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
status	order_count	total_revenue			
Delivered	6	17191.00			
Shipped	2	13596.00			
Cancelled	1	2999.00			
Pending	1	1299.00			

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```
149 • SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
150 FROM products GROUP BY category;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
category	most_expensive	cheapest			
Clothing	4599.00	799.00			
Electronics	3499.00	899.00			
Home	1299.00	599.00			

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

```
152 • SELECT category, AVG(unit_price) AS average_price
153 FROM products GROUP BY category
154 HAVING AVG(unit_price) > 2000;
```

Result Grid			Filter Rows:	Export:	Wrap Cell C
category	average_price				
Clothing	2699.000000				
Electronics	2224.000000				